

Category Theory and Functional Programming

Functors

Andrés Sicard-Ramírez

Universidad EAFIT

Semestre 2022-2

(Last modification: 5th May 2026)

Preliminaries

Textbook

S. Abramsky and N. Tzevelekos (2011). Introduction to Categories and Categorical Logic.

Convention

The numbers and page numbers assigned to chapters, examples, exercises, figures, quotes, sections and theorems on these slides correspond to the numbers assigned in the textbook.

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Introduction

Question

What about of morphisms between categories?

Introduction

Question

What about of morphisms between categories?

Answer

Of course, them are functors.

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Definition of a Functor

Definition

A **(covariant) functor** $F : \mathcal{C} \rightarrow \mathcal{D}$ between categories $\mathcal{C}$ and $\mathcal{D}$ is a mapping of objects to objects and arrows to arrows, that is,[†]

$$\begin{aligned} F_0 : \text{Obj}(\mathcal{C}) &\rightarrow \text{Obj}(\mathcal{D}) && \text{(object-map),} \\ F_1 : \text{Ar}(\mathcal{C}) &\rightarrow \text{Ar}(\mathcal{D}) && \text{(arrow-map),} \end{aligned}$$

which for all objects A, B and C in $\text{Obj}(\mathcal{C})$ and for all arrows $A \xrightarrow{f} B$ and $B \xrightarrow{g} C$ in $\text{Ar}(\mathcal{C})$, satisfies the **functoriality conditions**

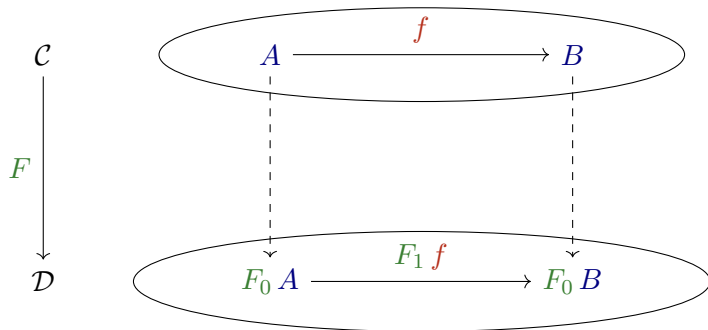
$$\begin{aligned} F_1(g \circ f) &= (F_1 g) \circ (F_1 f) && \text{(preservation of compositions),} \\ F_1 \text{id}_A &= \text{id}_{(F_0 A)} && \text{(preservation of identities).} \end{aligned}$$

[†]The textbook does not use F_0 and F_1 but F .

Definition of a Functor

Remark

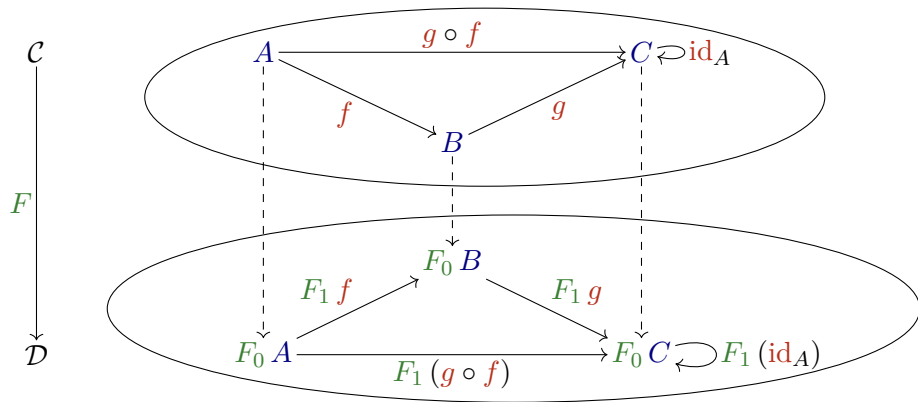
The functor $F : \mathcal{C} \rightarrow \mathcal{D}$ maps objects and arrows of $\mathcal{C}$ to objects and arrows of $\mathcal{D}$, respectively.



Definition of a Functor

Remark

The functor $F : \mathcal{C} \rightarrow \mathcal{D}$ preserves domains and codomains, identity arrows, and composition. It also maps each commutative diagram in $\mathcal{C}$ into a commutative diagram in $\mathcal{D}$.



Definition of a Functor

Remark

Given a functor $F : \mathcal{C} \rightarrow \mathcal{D}$, that is,

$$F_0 : \text{Obj}(\mathcal{C}) \rightarrow \text{Obj}(\mathcal{D}),$$

$$F_1 : \text{Ar}(\mathcal{C}) \rightarrow \text{Ar}(\mathcal{D}),$$

for all A, B in $\text{Obj}(\mathcal{C})$, there is the map

$$F_{A,B} : \mathcal{C}(A, B) \rightarrow \mathcal{D}(F_0 A, F_0 B),$$

and for all $f : A \rightarrow B$,

$$F_{A,B} f : F_0 A \rightarrow F_0 B.$$

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Examples of Functors

Example

Let $\mathcal{P}S$ be the power set of the set S . The **(covariant) power set functor**

$P : \mathbf{Set} \rightarrow \mathbf{Set}$, is defined by

Examples of Functors

Example

Let $\mathcal{P} S$ be the power set of the set S . The **(covariant) power set functor**

$P : \mathbf{Set} \rightarrow \mathbf{Set}$, is defined by

$$P_0 : \mathbf{Obj}(\mathbf{Set}) \rightarrow \mathbf{Obj}(\mathbf{Set})$$

$$P_0 X := \mathcal{P} X$$

Examples of Functors

Example

Let $\mathcal{P}S$ be the power set of the set S . The **(covariant) power set functor**

$P : \mathbf{Set} \rightarrow \mathbf{Set}$, is defined by

$$P_0 : \mathbf{Obj}(\mathbf{Set}) \rightarrow \mathbf{Obj}(\mathbf{Set})$$

$$P_0 X := \mathcal{P} X$$

$$P_1 : \mathbf{Ar}(\mathbf{Set}) \rightarrow \mathbf{Ar}(\mathbf{Set})$$

$$P_{x,y} : \mathbf{Set}(X, Y) \rightarrow \mathbf{Set}(P_0 X, P_0 Y)$$

$$P_{x,y} f : \mathcal{P} X \rightarrow \mathcal{P} Y$$

$$P_{x,y} f S := f(S) = \{ f(x) \mid x \in S \}$$

(continued on next slide)

Examples of Functors

Example (continuation)

Let $X = \{0, 1\}$, $Y = \{\emptyset, X\}$ and $f : X \rightarrow Y$ defined by $f(0) = \emptyset$ and $f(1) = X$. Then,

$$P_0 : \text{Obj}(\mathbf{Set}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$P_0 X := \mathcal{P} X = \{\emptyset, \{0\}, \{1\}, X\},$$

$$P_{X,Y} f : \mathcal{P} X \rightarrow \mathcal{P} Y$$

$$P_{X,Y} f \emptyset := f(\emptyset) = \emptyset,$$

$$P_{X,Y} f \{0\} := f(\{0\}) = \{\emptyset\},$$

$$P_{X,Y} f \{1\} := f(\{1\}) = \{X\},$$

$$P_{X,Y} f \{0, 1\} := f(\{0, 1\}) = \{\emptyset, X\}.$$

Examples of Functors

Example

Let $(P, \preceq)$ and $(Q, \preceq)$ be two pre-orders seen as categories, denoted $\mathcal{P}$ and $\mathcal{Q}$, respectively. A functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is defined by

Examples of Functors

Example

Let $(P, \preceq)$ and $(Q, \preceq)$ be two pre-orders seen as categories, denoted $\mathcal{P}$ and $\mathcal{Q}$, respectively. A functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is defined by

$$F_0 : \text{Obj}(\mathcal{P}) \rightarrow \text{Obj}(\mathcal{Q})$$

Examples of Functors

Example

Let $(P, \preceq)$ and $(Q, \preceq)$ be two pre-orders seen as categories, denoted $\mathcal{P}$ and $\mathcal{Q}$, respectively. A functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is defined by

$$F_0 : \text{Obj}(\mathcal{P}) \rightarrow \text{Obj}(\mathcal{Q})$$

$$F_1 : \text{Ar}(\mathcal{P}) \rightarrow \text{Ar}(\mathcal{Q})$$

$$F_{A,B} : \mathcal{P}(A, B) \rightarrow \mathcal{Q}(F_0 A, F_0 B)$$

$$F_{A,B} f : F_0 A \rightarrow F_0 B$$

Examples of Functors

Example

Let $(P, \preceq)$ and $(Q, \preceq)$ be two pre-orders seen as categories, denoted $\mathcal{P}$ and $\mathcal{Q}$, respectively. A functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is defined by

$$F_0 : \text{Obj}(\mathcal{P}) \rightarrow \text{Obj}(\mathcal{Q})$$

$$F_1 : \text{Ar}(\mathcal{P}) \rightarrow \text{Ar}(\mathcal{Q})$$

$$F_{A,B} : \mathcal{P}(A, B) \rightarrow \mathcal{Q}(F_0 A, F_0 B)$$

$$F_{A,B} f : F_0 A \rightarrow F_0 B$$

Since $\mathcal{P}(A, B)$ and $\mathcal{Q}(F_0 A, F_0 B)$ have at most an arrow, the map $F_{A,B}$ exists iff

$$A \preceq B \quad \text{implies} \quad F_0 A \preceq F_0 B.$$

Examples of Functors

Example

Let $(P, \preceq)$ and $(Q, \preceq)$ be two pre-orders seen as categories, denoted $\mathcal{P}$ and $\mathcal{Q}$, respectively. A functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is defined by

$$F_0 : \text{Obj}(\mathcal{P}) \rightarrow \text{Obj}(\mathcal{Q})$$

$$F_1 : \text{Ar}(\mathcal{P}) \rightarrow \text{Ar}(\mathcal{Q})$$

$$F_{A,B} : \mathcal{P}(A, B) \rightarrow \mathcal{Q}(F_0 A, F_0 B)$$

$$F_{A,B} f : F_0 A \rightarrow F_0 B$$

Since $\mathcal{P}(A, B)$ and $\mathcal{Q}(F_0 A, F_0 B)$ have at most an arrow, the map $F_{A,B}$ exists iff

$$A \preceq B \text{ implies } F_0 A \preceq F_0 B.$$

That is, a functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is just a monotone map which sends, if exists, the unique arrow $A \rightarrow B$ to the unique arrow $F_0 A \rightarrow F_0 B$.

Examples of Functors

Example

Let $(P, \preceq)$ and $(Q, \preceq)$ be two pre-orders seen as categories, denoted $\mathcal{P}$ and $\mathcal{Q}$, respectively. A functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is defined by

$$F_0 : \text{Obj}(\mathcal{P}) \rightarrow \text{Obj}(\mathcal{Q})$$

$$F_1 : \text{Ar}(\mathcal{P}) \rightarrow \text{Ar}(\mathcal{Q})$$

$$F_{A,B} : \mathcal{P}(A, B) \rightarrow \mathcal{Q}(F_0 A, F_0 B)$$

$$F_{A,B} f : F_0 A \rightarrow F_0 B$$

Since $\mathcal{P}(A, B)$ and $\mathcal{Q}(F_0 A, F_0 B)$ have at most an arrow, the map $F_{A,B}$ exists iff

$$A \preceq B \text{ implies } F_0 A \preceq F_0 B.$$

That is, a functor $F : \mathcal{P} \rightarrow \mathcal{Q}$ is just a monotone map which sends, if exists, the unique arrow $A \rightarrow B$ to the unique arrow $F_0 A \rightarrow F_0 B$.

Example from (Fong, Milewski and Spivak 2020, § 3.2.2).

Examples of Functors

Example

Let $(M, \cdot, \epsilon)$ and $(N, \diamond, \mu)$ be two monoids seen as categories, denoted $\mathcal{M}$ and $\mathcal{N}$, respectively. Let $*$ be the only object in both categories. A functor $F : \mathcal{M} \rightarrow \mathcal{N}$ is defined by

Examples of Functors

Example

Let $(M, \cdot, \epsilon)$ and $(N, \diamond, \mu)$ be two monoids seen as categories, denoted $\mathcal{M}$ and $\mathcal{N}$, respectively. Let $*$ be the only object in both categories. A functor $F : \mathcal{M} \rightarrow \mathcal{N}$ is defined by

$$F_0 : \text{Obj}(\mathcal{M}) \rightarrow \text{Obj}(\mathcal{N})$$

$$F_0 : \{*\} \rightarrow \{*\}$$

$$F_0 * = *$$

Examples of Functors

Example

Let $(M, \cdot, \epsilon)$ and $(N, \diamond, \mu)$ be two monoids seen as categories, denoted $\mathcal{M}$ and $\mathcal{N}$, respectively. Let $*$ be the only object in both categories. A functor $F : \mathcal{M} \rightarrow \mathcal{N}$ is defined by

$$F_0 : \text{Obj}(\mathcal{M}) \rightarrow \text{Obj}(\mathcal{N})$$

$$F_0 : \{*\} \rightarrow \{*\}$$

$$F_0 * = *$$

$$F_1 : \text{Ar}(\mathcal{M}) \rightarrow \text{Ar}(\mathcal{N})$$

$$F_{*,*} : \mathcal{P}(*,*) \rightarrow \mathcal{Q}(F_0 *, F_0 *)$$

$$F_{*,*} f : * \rightarrow *$$

Examples of Functors

Example

Let $(M, \cdot, \epsilon)$ and $(N, \diamond, \mu)$ be two monoids seen as categories, denoted $\mathcal{M}$ and $\mathcal{N}$, respectively. Let $*$ be the only object in both categories. A functor $F : \mathcal{M} \rightarrow \mathcal{N}$ is defined by

$$F_0 : \text{Obj}(\mathcal{M}) \rightarrow \text{Obj}(\mathcal{N})$$

$$F_0 : \{*\} \rightarrow \{*\}$$

$$F_0 * = *$$

$$F_1 : \text{Ar}(\mathcal{M}) \rightarrow \text{Ar}(\mathcal{N})$$

$$F_{*,*} : \mathcal{P}(*, *) \rightarrow \mathcal{Q}(F_0 *, F_0 *)$$

$$F_{*,*} f : * \rightarrow *$$

The functor F must satisfies:

$$F_{*,*} (m_1 \cdot m_2) = (F_{*,*} m_1) \diamond (F_{*,*} m_2), \quad \text{for all } m_1, m_2 \text{ in } \mathcal{M},$$

$$F_{*,*} \epsilon = \mu.$$

Examples of Functors

Example

Let $(M, \cdot, \epsilon)$ and $(N, \diamond, \mu)$ be two monoids seen as categories, denoted $\mathcal{M}$ and $\mathcal{N}$, respectively. Let $*$ be the only object in both categories. A functor $F : \mathcal{M} \rightarrow \mathcal{N}$ is defined by

$$F_0 : \text{Obj}(\mathcal{M}) \rightarrow \text{Obj}(\mathcal{N})$$

$$F_0 : \{*\} \rightarrow \{*\}$$

$$F_0 * = *$$

$$F_1 : \text{Ar}(\mathcal{M}) \rightarrow \text{Ar}(\mathcal{N})$$

$$F_{*,*} : \mathcal{P}(*, *) \rightarrow \mathcal{Q}(F_0 *, F_0 *)$$

$$F_{*,*} f : * \rightarrow *$$

The functor F must satisfy:

$$F_{*,*} (m_1 \cdot m_2) = (F_{*,*} m_1) \diamond (F_{*,*} m_2), \quad \text{for all } m_1, m_2 \text{ in } \mathcal{M},$$

$$F_{*,*} \epsilon = \mu.$$

That is, a functor $F : \mathcal{M} \rightarrow \mathcal{N}$ is just a monoid homomorphism.

Examples of Functors

Example

The **identity functor** $\text{Id}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathcal{C}$ in a category $\mathcal{C}$ is the functor that maps each object and each arrow of $\mathcal{C}$ to itself.

Examples of Functors

Example

Let $F : \mathbf{Mon} \rightarrow \mathbf{Set}$ be the **forgetful functor** which

- (i) sends a monoid to its set of elements and
- (ii) sends a homomorphism between monoids to the corresponding function between sets.

Examples of Functors

Example

Let $[S]$ be the set of all finite lists of elements of S . The **list functor**

$\text{List} : \mathbf{Set} \rightarrow \mathbf{Set}$, is defined by

Examples of Functors

Example

Let $[S]$ be the set of all finite lists of elements of S . The **list functor**

$\text{List} : \mathbf{Set} \rightarrow \mathbf{Set}$, is defined by

$\text{List}_0 : \text{Obj}(\mathbf{Set}) \rightarrow \text{Obj}(\mathbf{Set})$

$\text{List}_0 X := [X]$

Examples of Functors

Example

Let $[S]$ be the set of all finite lists of elements of S . The **list functor**

$\text{List} : \mathbf{Set} \rightarrow \mathbf{Set}$, is defined by

$$\text{List}_0 : \text{Obj}(\mathbf{Set}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$\text{List}_1 : \text{Ar}(\mathbf{Set}) \rightarrow \text{Ar}(\mathbf{Set})$$

$$\text{List}_0 X := [X]$$

$$\text{List}_{X,Y} : \mathbf{Set}(X, Y) \rightarrow \mathbf{Set}(\text{List}_0 X, \text{List}_0 Y)$$

$$\text{List}_{X,Y} f : [X] \rightarrow [Y]$$

$$\text{List}_{X,Y} f [x_1, x_2, \dots, x_n] := [f(x_1), f(x_2), \dots, f(x_n)]$$

Examples of Functors

Example

The **free monoid functor** $\mathbf{MList} : \mathbf{Set} \rightarrow \mathbf{Mon}$ maps every set X to the free monoid over X .

Examples of Functors

Example

The **free monoid functor** $\mathbf{MList} : \mathbf{Set} \rightarrow \mathbf{Mon}$ maps every set X to the free monoid over X .

Let $(-) * (-)$ be the list concatenation function and let ε be the empty list, the functor is defined by

Examples of Functors

Example

The **free monoid functor** $\mathbf{MList} : \mathbf{Set} \rightarrow \mathbf{Mon}$ maps every set X to the free monoid over X .

Let $(-)*(-)$ be the list concatenation function and let ε be the empty list, the functor is defined by

$$\mathbf{MList}_0 : \mathbf{Obj}(\mathbf{Set}) \rightarrow \mathbf{Obj}(\mathbf{Mon})$$

$$\mathbf{MList}_0 X := (\mathbf{List}_0 X, *, \varepsilon) = ([X], *, \varepsilon)$$

$$\mathbf{MList}_1 : \mathbf{Ar}(\mathbf{Set}) \rightarrow \mathbf{Ar}(\mathbf{Mon})$$

$$\mathbf{MList}_{X,Y} : \mathbf{Set}(X, Y) \rightarrow \mathbf{Mon}(\mathbf{MList}_0 X, \mathbf{MList}_0 Y)$$

$$\mathbf{MList}_{X,Y} f : ([X], *, \varepsilon) \rightarrow ([Y], *, \varepsilon)$$

$$\mathbf{MList}_{X,Y} f [x_1, x_2, \dots, x_n] := \mathbf{List}_{X,Y} f [x_1, x_2, \dots, x_n]$$

Exercises

Exercise

Verify that functors $F : \mathbf{2}_{\Rightarrow} \rightarrow \mathbf{Set}$ correspond to directed graphs (textbook, Exercise 45).

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Functors in Haskell

Introduction via Maybe
(Whiteboard).

Functors in Haskell

Introduction via Maybe
(Whiteboard).

The typeclass Functor

```
class Functor f where  
    fmap :: (a -> b) -> f a -> f b
```

Functors in Haskell

Example

The polymorphic type constructor `Maybe` is a functor whose instance is defined by

```
instance Functor Maybe where
    fmap _ Nothing  = Nothing
    fmap f (Just a) = Just (f a)
```

Functors in Haskell

Example

The polymorphic type constructor `Maybe` is a functor whose instance is defined by

```
instance Functor Maybe where
  fmap _ Nothing  = Nothing
  fmap f (Just a) = Just (f a)
```

Exercise

Show that the `Maybe` functor satisfies the functoriality conditions.

Functors in Haskell

Example

`ReadInt` is a type constructor that turns any type `a` into a new type that reads a value of `Int` to create a value of `a` (Fong, Milewski and Spivak 2020, Example 3.41).

```
data ReadInt a = MkReadInt (Int -> a)
```

Functors in Haskell

Example

`ReadInt` is a type constructor that turns any type `a` into a new type that reads a value of `Int` to create a value of `a` (Fong, Milewski and Spivak 2020, Example 3.41).

```
data ReadInt a = MkReadInt (Int -> a)
```

`ReadInt` is a functor via the following instance.

```
instance Functor ReadInt where
  fmap f (MkReadInt g) = MkReadInt (f . g)
```

Functors in Haskell

Example

The (binary) function type $(\rightarrow) :: a \rightarrow b \rightarrow (a \rightarrow b)$ is a functor.

```
instance Functor ((->) a) where
    fmap f g = f . g
```

Note that $\text{fmap} :: (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow (a \rightarrow c)$.

Functors in Haskell

Exercise

To define an instance of `Functor` for the (binary) product type

`(,) :: a -> b -> (a,b).`

Functors in Haskell

Example

Recall that terminal object (unit type) in [Haskell](#) is `() :: ()`. We can define a constant functor by

```
data CUnit a = MkCU ()

instance Functor CUnit where
    fmap f (MkCU ()) = MkCU ()
```

Functors in Haskell

Exercise

Given a constant “functor” defined by

```
data CBool a = MkCB Bool

instance Functor CBool where
    fmap f (MkCB True)  = MkCB False
    fmap f (MkCB False) = MkCB True
```

Is CBool really a functor?

Functors in Haskell

Exercise

We define a constant functor by

```
data CInt a = MkCI Int
```

Show that the polymorphic type constructor `CInt` can be given the structure of a functor by saying how it lifts morphisms. That is, provide a [Haskell](#) function `mapCInt` of the type $(a \rightarrow b) \rightarrow (CInt\ a \rightarrow CInt\ b)$ (Fong, Milewski and Spivak 2020, Exercise 3.46).

Functors in Haskell

Exercise

For each of the following type constructors, define two versions of `fmap`, one of which has a corresponding functor `Hask` $\rightarrow$ `Hask`, and one of which does not (Fong, Milewski and Spivak 2020, Exercise 3.48).

- (i) `data WithString a = WithStr (a, String)`
- (ii) `data ConstStr a = ConstStr String`
- (iii) `data List a = Nil | Cons (a, List a)`

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

The Product Category

Definition

Let $\mathcal{C}$ and $\mathcal{D}$ be two categories. The **product category** $\mathcal{C} \times \mathcal{D}$ is defined by:

- (i) Objects: (C, D) , where C and D are objects in $\mathcal{C}$ and $\mathcal{D}$, respectively.
- (ii) Arrows: $(C, D) \xrightarrow{(f, g)} (C', D')$, where $C \xrightarrow{f} C'$ and $D \xrightarrow{g} D'$ are arrows in $\mathcal{C}$ and $\mathcal{D}$, respectively.
- (iii) Composition
- (iv) Identities

$$(f', g') \circ (f, g) := (f' \circ f, g' \circ g).$$

$$\text{id}_{(C, D)} := (\text{id}_C, \text{id}_D).$$

Definition of a Binary Functor

Definition

Let $\mathcal{C}$, $\mathcal{D}$ and $\mathcal{E}$ be three categories. A **binary functor** (or **bifunctor**) is a functor whose domain is a product category, that is, a binary functor from $\mathcal{C} \times \mathcal{D}$ to $\mathcal{E}$ is a functor

$$F : \mathcal{C} \times \mathcal{D} \rightarrow \mathcal{E}.$$

Example of Binary Functors

Example

The projection functors $\mathcal{C} \xleftarrow{\pi_1} \mathcal{C} \times \mathcal{D} \xrightarrow{\pi_2} \mathcal{D}$ are binary functors.

Example of Binary Functors

Example

The projection functors $\mathcal{C} \xleftarrow{\pi_1} \mathcal{C} \times \mathcal{D} \xrightarrow{\pi_2} \mathcal{D}$ are binary functors.

(i) For $\pi_1 : \mathcal{C} \times \mathcal{D} \rightarrow \mathcal{C}$ we have:

$$(\pi_1)_0 : \text{Obj}(\mathcal{C} \times \mathcal{D}) \rightarrow \text{Obj}(\mathcal{C})$$

$$(\pi_1)_0 (C, D) := C$$

$$(\pi_1)_1 : \text{Ar}(\mathcal{C} \times \mathcal{D}) \rightarrow \text{Ar}(\mathcal{C})$$

$$(\pi_1)_{(C,D),(C',D')} : \text{Mor}_{\mathcal{C} \times \mathcal{D}}((C, D), (C', D')) \rightarrow \text{Mor}_{\mathcal{C}}((\pi_1)_0 (C, D), (\pi_1)_0 (C', D')),$$

$$(\pi_1)_{(C,D),(C',D')} (f, g) : C \rightarrow C'$$

$$(\pi_1)_{(C,D),(C',D')} (f, g) := f$$

Example of Binary Functors

Example

The projection functors $\mathcal{C} \xleftarrow{\pi_1} \mathcal{C} \times \mathcal{D} \xrightarrow{\pi_2} \mathcal{D}$ are binary functors.

(i) For $\pi_1 : \mathcal{C} \times \mathcal{D} \rightarrow \mathcal{C}$ we have:

$$(\pi_1)_0 : \text{Obj}(\mathcal{C} \times \mathcal{D}) \rightarrow \text{Obj}(\mathcal{C})$$

$$(\pi_1)_0(C, D) := C$$

$$(\pi_1)_1 : \text{Ar}(\mathcal{C} \times \mathcal{D}) \rightarrow \text{Ar}(\mathcal{C})$$

$$(\pi_1)_{(C,D),(C',D')} : \text{Mor}_{\mathcal{C} \times \mathcal{D}}((C, D), (C', D')) \rightarrow \text{Mor}_{\mathcal{C}}((\pi_1)_0(C, D), (\pi_1)_0(C', D')),$$

$$(\pi_1)_{(C,D),(C',D')}(\textcolor{red}{f}, \textcolor{red}{g}) : C \rightarrow C'$$

$$(\pi_1)_{(C,D),(C',D')}(\textcolor{red}{f}, \textcolor{red}{g}) := \textcolor{red}{f}$$

(ii) Similarly for $\pi_2 : \mathcal{C} \times \mathcal{D} \rightarrow \mathcal{D}$.

The Product Functor

Definition

Let $\mathcal{C}$ be a category with binaries products, and let $\mathcal{C} \times \mathcal{C}$ be the product category of $\mathcal{C}$ with itself. The **product functor** $\times : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$ is a binary functor defined by

The Product Functor

Definition

Let $\mathcal{C}$ be a category with binary products, and let $\mathcal{C} \times \mathcal{C}$ be the product category of $\mathcal{C}$ with itself. The **product functor** $\times : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$ is a binary functor defined by

$$\times_0 : \text{Obj}(\mathcal{C} \times \mathcal{C}) \rightarrow \text{Obj}(\mathcal{C})$$

$$\times_0(A, B) := A \times B \text{ (binary product)}$$

$$\times_1 : \text{Ar}(\mathcal{C} \times \mathcal{C}) \rightarrow \text{Ar}(\mathcal{C})$$

$$\times_{(A, A'), (B, B')} : \mathcal{C} \times \mathcal{C}((A, A'), (B, B')) \rightarrow \mathcal{C}(\times_0(A, A'), \times_0(B, B'))$$

$$\times_1(f : A \rightarrow B, g : A' \rightarrow B') : A \times A' \rightarrow B \times B'$$

$$\times_1(f : A \rightarrow B, g : A' \rightarrow B') := f \times g \text{ (product morphism)}$$

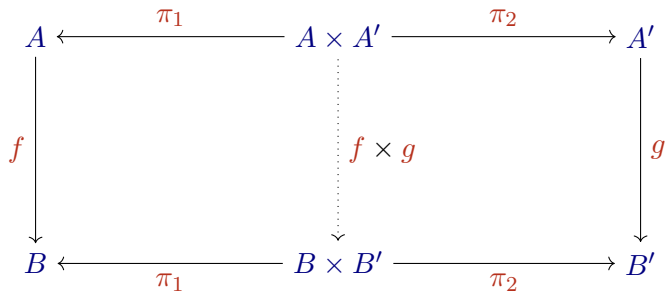
where $f \times g := \langle f \circ \pi_1, g \circ \pi_2 \rangle$.

(continued on next slide)

The Product Functor

Definition (continuation)

That is, both squares in the following diagram commute.



$$\begin{pmatrix} f \circ \pi_1 = \pi_1 \circ (f \times g) \\ g \circ \pi_2 = \pi_2 \circ (f \times g) \end{pmatrix}$$

N -Ary Functors

Remark

Binary functors can be generalised to n -ary functors.

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Small and Large Categories

Introduction

Before defining a category of categories, we need to classify the categories in small and large for avoiding that it be an object of itself.

Small and Large Categories

Definition

A category is **small** iff **both** the collection of its objects and the collection of its arrows are **sets**. Otherwise, the category is **large** (Awodey 2010).

Small and Large Categories

Example

The finite categories $\mathbf{1}, \mathbf{2}, \dots, \mathbf{n}$, a monoid viewed as a category, and a pre-order viewed as a category are small categories.

Small and Large Categories

Example

The finite categories $\mathbf{1}, \mathbf{2}, \dots, \mathbf{n}$, a monoid viewed as a category, and a pre-order viewed as a category are small categories.

Example

The categories **Set**, **Pos**, **Mon**, **Grp** and **Top** are large categories.

Locally Small Categories

Definition

A category $\mathcal{C}$ is **locally small** iff for all objects A, B the collection $\mathcal{C}(A, B)$ is a **set** (Awodey 2010).

Locally Small Categories

Definition

A category $\mathcal{C}$ is **locally small** iff for all objects A, B the collection $\mathcal{C}(A, B)$ is a **set** (Awodey 2010).

Remark

- Recall from the previous conventions that if the collection $\mathcal{C}(A, B)$ is a set it is called a hom-set and it is denoted $\text{hom}_{\mathcal{C}}(A, B)$.
- Also recall that in the textbook all the collections $\mathcal{C}(A, B)$ are hom-sets.

Locally Small Categories

Example

Any small category is locally small.

Locally Small Categories

Example

Any small category is locally small.

Example

The categories **Set**, **Pos**, **Mon**, **Grp** and **Top** are locally small categories.

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

The Category of Small Categories

Definition

The category $\mathbf{Cat}$ is the category of small categories:

- (i) Objects: Small categories
- (ii) Arrows: Functors

(continued on next slide)

The Category of Small Categories

Definition (continuation)

(iii) Composition of functors

Let $\mathcal{C}$, $\mathcal{D}$ and $\mathcal{E}$ be small categories and let $F : \mathcal{C} \rightarrow \mathcal{D}$ and $G : \mathcal{D} \rightarrow \mathcal{E}$ be two functors, then

$$G \circ F : \mathcal{C} \rightarrow \mathcal{E},$$

$$(G \circ F)_0 : \text{Obj}(\mathcal{C}) \rightarrow \text{Obj}(\mathcal{E})$$

$$(G \circ F)_0 A := G_0 (F_0 A),$$

$$(G \circ F)_1 : \text{Ar}(\mathcal{C}) \rightarrow \text{Ar}(\mathcal{E}),$$

$$(G \circ F)_{A,B} : \mathcal{C}(A, B) \rightarrow \mathcal{E}((G \circ F)_0 A, (G \circ F)_0 B)$$

$$(G \circ F)_{A,B} f : G_0 (F_0 A) \rightarrow G_0 (F_0 B)$$

$$(G \circ F)_{A,B} f := G_1 (F_1 f).$$

(continued on next slide)

The Category of Small Categories

Definition (continuation)

(iii) Composition of functors

That is,

$$G \circ F : \mathcal{C} \rightarrow \mathcal{E} := \begin{cases} A \mapsto G(F A), \\ f \mapsto G(F f). \end{cases}$$

The Category of Small Categories

Definition (continuation)

(iii) Composition of functors

That is,

$$G \circ F : \mathcal{C} \rightarrow \mathcal{E} := \begin{cases} A \mapsto G(F A), \\ f \mapsto G(F f). \end{cases}$$

(iv) Identity functors

Let $\mathcal{C}$ be a small category, then

$$\text{id}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathcal{C} := \begin{cases} A \mapsto A, \\ f \mapsto f. \end{cases}$$

The Category of Small Categories

Remark

The category **Cat** is large and therefore it is not object of itself.

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Introduction

Description

A **covariant** functor F preserves the direction of arrows, that is,

$$F_1 (f : A \rightarrow B) : F_0 A \rightarrow F_0 B.$$

A **contravariant** functor G reverses the direction of arrows, that is,

$$G_1 (f : A \rightarrow B) : G_0 B \rightarrow G_0 A.$$

Contravariant Functors

Definition

Let $\mathcal{C}$ and $\mathcal{D}$ be two categories. A **contravariant functor** G from $\mathcal{C}$ to $\mathcal{D}$ is a functor

$$G : \mathcal{C}^{\text{op}} \rightarrow \mathcal{D} \quad (\text{or } \mathcal{C} \rightarrow \mathcal{D}^{\text{op}}),$$

$$G_0 : \text{Obj}(\mathcal{C}^{\text{op}}) \rightarrow \text{Obj}(\mathcal{D}) \quad (\text{object-map}),$$

$$G_1 : \text{Ar}(\mathcal{C}^{\text{op}}) \rightarrow \text{Ar}(\mathcal{D}) \quad (\text{arrow-map}),$$

$$G_{A,B} : \mathcal{C}^{\text{op}}(A, B) \rightarrow \mathcal{D}(G_0 B, G_0 A)$$

$$G_{A,B} f : G_0 B \rightarrow G_0 A,$$

which for all objects A, B and C in $\text{Obj}(\mathcal{C}^{\text{op}})$ and for all arrows $A \xrightarrow{f} B$ and $B \xrightarrow{g} C$ in $\text{Ar}(\mathcal{C}^{\text{op}})$, satisfies the **functoriality conditions**

$$G_1(g \circ f) = (G_1 f) \circ (G_1 g) \quad (\text{preservation of compositions}),$$

$$G_1(\text{id}_A) = \text{id}_{(G_0 A)} \quad (\text{preservation of identities}).$$

Contravariant Functors

Example

Let $\mathcal{P}S$ be the power set of the set S . The **contravariant power set functor**

$\mathcal{P}^{\text{op}} : \mathbf{Set}^{\text{op}} \rightarrow \mathbf{Set}$, is defined by

Contravariant Functors

Example

Let $\mathcal{P} S$ be the power set of the set S . The **contravariant power set functor**

$P^{\text{op}} : \mathbf{Set}^{\text{op}} \rightarrow \mathbf{Set}$, is defined by

$$P_0^{\text{op}} : \text{Obj}(\mathbf{Set}^{\text{op}}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$P_0^{\text{op}} X := \mathcal{P} X$$

Contravariant Functors

Example

Let $\mathcal{P}S$ be the power set of the set S . The **contravariant power set functor**

$P^{\text{op}} : \mathbf{Set}^{\text{op}} \rightarrow \mathbf{Set}$, is defined by

$$P_0^{\text{op}} : \text{Obj}(\mathbf{Set}^{\text{op}}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$P_0^{\text{op}} X := \mathcal{P} X$$

$$P_1^{\text{op}} : \text{Ar}(\mathbf{Set}^{\text{op}}) \rightarrow \text{Ar}(\mathbf{Set})$$

$$P_{x,y}^{\text{op}} : \mathbf{Set}^{\text{op}}(X, Y) \rightarrow \mathbf{Set}(P_0^{\text{op}} Y, P_0^{\text{op}} X)$$

$$P_{x,y}^{\text{op}} f : \mathcal{P} Y \rightarrow \mathcal{P} X$$

$$P_{x,y}^{\text{op}} f T := f^{-1}(T) = \{ x \in X \mid f(x) \in T \}$$

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Hom-Functors

Definition (first notation)

Let $\mathcal{C}$ be a locally small category and let A be an object of $\mathcal{C}$. The **covariant Set-valued hom-functor** $\mathcal{C}(A, -)$ is defined by

$$\mathcal{C}(A, -) : \mathcal{C} \rightarrow \mathbf{Set},$$

$$\mathcal{C}(A, -)_0 : \mathrm{Obj}(\mathcal{C}) \rightarrow \mathrm{Obj}(\mathbf{Set})$$

$$\mathcal{C}(A, C)_0 := \mathcal{C}(A, C),$$

$$\mathcal{C}(A, -)_1 : \mathrm{Ar}(\mathcal{C}) \rightarrow \mathrm{Ar}(\mathbf{Set})$$

$$\mathcal{C}(A, -)_{C,D} : \mathcal{C}(C, D) \rightarrow \mathbf{Set}(\mathcal{C}(A, -)_0 C, \mathcal{C}(A, -)_0 D)$$

$$\mathcal{C}(A, f)_{C,D} : \mathcal{C}(A, C) \rightarrow \mathcal{C}(A, D)$$

$$\mathcal{C}(A, f)_{C,D} g := f \circ g.$$

Hom-Functors

Definition (first notation)

Let $\mathcal{C}$ be a (locally small) category and let B be an object of $\mathcal{C}$. The **contravariant Set-valued hom-functor** $\mathcal{C}(-, B)$ is defined by

$$\mathcal{C}(-, B) : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set},$$

$$\mathcal{C}(-, B)_0 : \text{Obj}(\mathcal{C}^{\text{op}}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$\mathcal{C}(C, B)_0 := \mathcal{C}(C, B),$$

$$\mathcal{C}(-, B)_1 : \text{Ar}(\mathcal{C}^{\text{op}}) \rightarrow \text{Ar}(\mathbf{Set})$$

$$\mathcal{C}(-, B)_{C,D} : \mathcal{C}^{\text{op}}(C, D) \rightarrow \mathbf{Set}(\mathcal{C}(-, B)_0 D, \mathcal{C}(-, B)_0 C)$$

$$\mathcal{C}(f, B)_{C,D} : \mathcal{C}(D, B) \rightarrow \mathcal{C}(C, B)$$

$$\mathcal{C}(f, B)_{C,D} g := g \circ f.$$

Hom-Functors

Exercise

Let $\mathcal{C}$ be a (locally small) category. Spell out the definition of the set-valued hom-functor $\mathcal{C}(-, -) : \mathcal{C}^{\text{op}} \times \mathcal{C} \rightarrow \mathbf{Set}$. Verify carefully that it is a functor (textbook, Exercise 47).

Hom-Functors

Notation

Recall that if $\mathcal{C}$ is a locally small category the collection of arrows of an object A to an object B is a **set** and it is denoted by $\text{hom}_{\mathcal{C}}(A, B)$, that is,

$$\text{hom}_{\mathcal{C}}(A, B) := \left\{ f \in \text{Ar}(\mathcal{C}) \mid A \xrightarrow{f} B \right\} =: \mathcal{C}(A, B).$$

Hom-Functors

Definition (second notation)

Let $\mathcal{C}$ be a locally small category and let A be an object of $\mathcal{C}$. The **covariant Set-valued hom-functor** $\text{hom}_{\mathcal{C}}(A, -)$ is defined by

$$\text{hom}_{\mathcal{C}}(A, -) : \mathcal{C} \rightarrow \mathbf{Set},$$

$$\text{hom}_{\mathcal{C}}(A, -)_0 : \text{Obj}(\mathcal{C}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$\text{hom}_{\mathcal{C}}(A, C)_0 := \text{hom}_{\mathcal{C}}(A, C)$$

$$\text{hom}_{\mathcal{C}}(A, -)_1 : \text{Ar}(\mathcal{C}) \rightarrow \text{Ar}(\mathbf{Set})$$

$$\text{hom}_{\mathcal{C}}(A, -)_{C,D} : \text{hom}_{\mathcal{C}}(C, D) \rightarrow \mathbf{Set}(\text{hom}_{\mathcal{C}}(A, C), \text{hom}_{\mathcal{C}}(A, D))$$

$$\text{hom}_{\mathcal{C}}(A, f : C \rightarrow D) : \text{hom}_{\mathcal{C}}(A, C) \rightarrow \text{hom}_{\mathcal{C}}(A, D)$$

$$\text{hom}_{\mathcal{C}}(A, f : C \rightarrow D) g := f \circ g$$

Hom-Functors

Definition (second notation)

Let $\mathcal{C}$ be a (locally small) category and let B be an object of $\mathcal{C}$. The **contravariant Set-valued hom-functor** $\text{hom}_{\mathcal{C}}(-, B)$ is defined by

$$\text{hom}_{\mathcal{C}}(-, B) : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set},$$

$$\text{hom}_{\mathcal{C}}(-, B)_0 : \text{Obj}(\mathcal{C}^{\text{op}}) \rightarrow \text{Obj}(\mathbf{Set})$$

$$\text{hom}_{\mathcal{C}}(C, B)_0 := \text{hom}_{\mathcal{C}}(C, B)$$

$$\text{hom}_{\mathcal{C}}(-, B)_1 : \text{Ar}(\mathcal{C}^{\text{op}}) \rightarrow \text{Ar}(\mathbf{Set})$$

$$\text{hom}_{\mathcal{C}}(-, B)_{C,D} : \text{hom}_{(\mathcal{C}^{\text{op}})}(C, D) \rightarrow \mathbf{Set}(\text{hom}_{\mathcal{C}}(D, B), \text{hom}_{\mathcal{C}}(C, B))$$

$$\text{hom}_{\mathcal{C}}(f : C \rightarrow D, B) : \text{hom}_{\mathcal{C}}(D, B) \rightarrow \text{hom}_{\mathcal{C}}(C, B)$$

$$\text{hom}_{\mathcal{C}}(f : C \rightarrow D, B) g := g \circ f$$

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

Faithful and Full Functors

Definition

Let $\mathcal{C}$ and $\mathcal{D}$ be (locally small) categories and let $F : \mathcal{C} \rightarrow \mathcal{D}$ be a functor.

- (i) The functor F is **faithful** iff each map $F_{A,B} : \mathcal{C}(A, B) \rightarrow \mathcal{D}(F_0 A, F_0 B)$ is injective.
- (ii) The functor F is **full** iff each map $F_{A,B} : \mathcal{C}(A, B) \rightarrow \mathcal{D}(F_0 A, F_0 B)$ is surjective.

Faithful and Full Functors

Example

The forgetful functor $F : \mathbf{Mon} \rightarrow \mathbf{Set}$ is faithful, but not full (explanation in the whiteboard).

Faithful and Full Functors

Example

The forgetful functor $F : \mathbf{Mon} \rightarrow \mathbf{Set}$ is faithful, but not full (explanation in the whiteboard).

Let $(M, \cdot, 1_M)$ and $(N, *, 1_N)$ be two monoids and let $f : M \rightarrow N$ be a homomorphism between them.

- Since $F_1 f = f$, the map F_1 is injective.
- If $g : M \rightarrow N$ is any function in $\mathbf{Set}$ such that $g(1_M) \neq 1_N$, then g is not a homomorphism between $(M, \cdot, 1_M)$ and $(N, *, 1_N)$. Therefore the map F_1 is not surjective.

Faithful and Full Functors

Exercise

Show that the free monoid functor $\mathbf{MList} : \mathbf{Set} \rightarrow \mathbf{Mon}$ is faithful, but not full.

Exercise

[1.3.5.2] Let $\mathcal{C}$ be a category with binary products such that, for each pair of objects A, B ,

$$\mathcal{C}(A, B) \neq \emptyset. \quad (*)$$

- (i) Show that the product functor $\times : \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C}$ is faithful.
- (ii) Would $-\times-$ still be faithful in the absence of condition $(*)$?

Preservation and Reflection

Definition

Let P be a property of arrows and let $F : \mathcal{C} \rightarrow \mathcal{D}$ be a functor.

- (i) The functor F **preserves** the property P iff
if f satisfies P then $F_1 f$ satisfies P .
- (ii) The functor F **reflects** the property P iff
if $F_1 f$ satisfies P then f satisfies P .

Preservation and Reflection

Example

Show that all functors preserve isomorphisms.

Preservation and Reflection

Example

Show that all functors preserve isomorphisms.

Example

Show that full and faithful functors reflect isomorphisms.

Outline

Introduction

Definition of a Functor

Examples of Functors

Functors in Haskell

Binary Functors

Small, Large and Locally Small Categories

The Category of Small Categories

Contravariance

Hom-Functors

Properties of Functors

References

References

- S. Abramsky and N. Tzevelekos (2011). Introduction to Categories and Categorical Logic. In: New Structures for Physics. Ed. by Bob Coecke. Vol. 813. Lecture Notes in Physics. Springer, pp. 3–94. DOI: [10.1007/978-3-642-12821-9_1](https://doi.org/10.1007/978-3-642-12821-9_1) (cit. on p. 2).
- Steve Awodey 2006 (2010). Category Theory. 2nd ed. Vol. 52. Oxford Logic Guides. Oxford University Press (cit. on pp. 62, 65, 66).
- Brendan Fong, Bartosz Milewski and David I. Spivak (2020). Programming with Categories (DRAFT). Last update: October 6, 2020. URL: <http://brendanfong.com/programmingcats.html> (cit. on pp. 17–22, 42, 43, 48, 49).